

Fiber Offers

Reusable, static, offline-capable payment requests for Fiber Network — the BOLT12 equivalent Fiber doesn't have yet.

Document	Product Requirements Document (PRD)
Category	Wallet & Payment UX Infrastructure
Author	Leo — Dilamme / Independent
Version	1.0
Status	Draft for Hackathon Submission

Table of Contents

- 1. 1. Executive Summary
- 2. 2. Problem Statement
- 3. 3. Goals & Objectives
- 4. 4. Target Users & Personas
- 5. 5. Product Overview
- 5. 6. Core Concepts
- 7. 7. User Stories & Use Cases
- 3. 8. Feature Set
- 9. 9. User Flows
- 0. 10. Success Metrics
- 1. 11. Two-Week Delivery Plan
- 2. 12. Hackathon Deliverables Mapping
- 3. 13. Risks & Mitigations
- 4. Appendix A: Competitive / Comparative Landscape
- 5. Appendix B: Glossary

1. Executive Summary

Fiber Offers is a reusable, static payment-request protocol for Fiber Network — CKB's Lightning-style payment channel network. It closes the single biggest UX gap between Fiber and Bitcoin's Lightning Network today: Fiber, like early Lightning, only supports single-use invoices. Every payment requires the recipient to be online, generate a fresh invoice, and share it out-of-band. Lightning solved this in 2024 with **BOLT12 "Offers"** — a reusable, publishable payment code that can be scanned once and paid many times, supports subscriptions natively, and improves payer/payee privacy. Fiber has no equivalent.

This project builds that missing primitive for Fiber: a protocol specification, a Rust/TypeScript reference implementation, a wallet SDK with drop-in UI components, and an optional human-readable "Fiber Address" resolver (the Lightning Address equivalent). It is designed to be immediately useful to three audiences at once — wallet builders, merchants, and Leo's own in-progress product, Loavix, a decentralized invoicing and billing system built on Fiber.

One-line pitch: Fiber currently forces every payment through a one-time invoice. Fiber Offers lets anyone publish a payment code once and get paid forever — unlocking subscriptions, tip jars, merchant checkout, and recurring billing on Fiber for the first time.

2. Problem Statement

2.1 What's broken today

Fiber's current invoice model mirrors Lightning's original BOLT11 design, which the Lightning community spent years working around:

Limitation	Practical impact
Invoices are single-use	A merchant or service must generate a brand-new invoice for every single payment. No static "pay me" QR code is possible.
Recipient must be online	If a node is briefly offline, it cannot issue an invoice, and no payment can be received — even though the underlying channel liquidity may be sitting there ready.
No native recurring payments	Subscriptions, recurring invoices, and pay-as-you-go billing require a custom scheduling layer built entirely outside the protocol.
No standard human-readable identifier	There is nothing equivalent to a Lightning Address (<code>user@domain</code>) for Fiber — users must copy-paste long encoded strings.
Fragmented, one-off integrations	Every app that wants "pay me anytime" behavior on Fiber today has to hand-roll its own workaround, none of which are interoperable.

2.2 Why this matters right now

The Fiber Network Infrastructure Hackathon explicitly calls out this exact gap under the Wallet & Payment UX Infrastructure category: "*Fiber payment request, invoice, QR, or payment intent formats for apps and wallets.*" Lightning's own ecosystem treats reusable offers as one of the most significant UX upgrades in the network's history — three of four major Lightning implementations (Core Lightning, LDK, Eclair) now support BOLT12 natively. Fiber has an opportunity to adopt this pattern early, before dozens of incompatible workarounds are built independently by different teams.

2.3 Why Leo/Loavix specifically

Loavix — a decentralized invoicing and billing system already being built on Fiber — needs exactly this capability: recurring, stablecoin-denominated invoices that don't require regenerating a payment request every billing cycle. Building Fiber Offers is not hackathon-only scope work; it is a real dependency for a real product already in progress, backed by existing wallet-signature auth (challenge/nonce/JWT with lock-hash ownership checks) and Fiber invoice-generation code.

3. Goals & Objectives

3.1 Primary goals

- Define an open, versioned **Fiber Offer** format: a static, reusable, signed payment identifier that can be published once (QR, link, NFC tag, printed code) and paid any number of times.
- Ship a reference **offer resolution service** that turns an Offer into a fresh, single-use Fiber invoice on demand — without requiring the payer to interact with anything beyond scanning a code.
- Ship a **wallet SDK** (TypeScript, React + React Native components) so any Fiber wallet or app can create, publish, resolve, and pay Offers with minimal integration work.
- Support **native recurrence** (e.g. "charge 10 USDI monthly") as a first-class field in the Offer schema, not a bolt-on.
- Ship a **Fiber Address** resolver (`user@domain` → Offer), the human-readable identity layer equivalent to Lightning Address / LNURL-pay.

3.2 Secondary goals

- Demonstrate the protocol live against Loavix's invoicing flow as a real integration, not just a toy demo.
- Make the reference implementation reusable by other CKB/Fiber teams with clear documentation and an open-source repo.

3.3 Non-goals (explicitly out of scope)

- This is **not** an on/off-ramp, fiat gateway, or KYC/compliance layer.
- This is **not** a full merchant checkout product (that's Category 3 scope) — Fiber Offers is the underlying primitive a checkout product would be built on.
- This is **not** a consensus-layer or protocol-level change to Fiber Network itself. It is an application/infrastructure layer that sits on top of existing Fiber node RPCs.
- Full BOLT12-style onion-message-native transport is a stretch goal, not a requirement for MVP — MVP may use an HTTP resolver (LNURL-style) as the transport, with a documented path to a fully native version.

4. Target Users & Personas

Wallet & App Developers

Building a Fiber-enabled wallet or app and need a "receive" flow that doesn't force users to regenerate invoices constantly. Want a drop-in SDK, not a protocol to design themselves.

Merchants & Service Providers

Want one static payment code for a storefront, tip jar, or subscription product. Currently blocked because Fiber invoices expire and can't be reused.

Fiber Node Operators

Run infrastructure and want to expose reusable receive endpoints without building custom invoice-issuing servers from scratch for every use case.

Loavix (internal / dogfooding user)

Needs recurring, stablecoin-denominated billing on Fiber today. Directly consumes the Offer + recurrence + Fiber Address features as production dependencies.

5. Product Overview

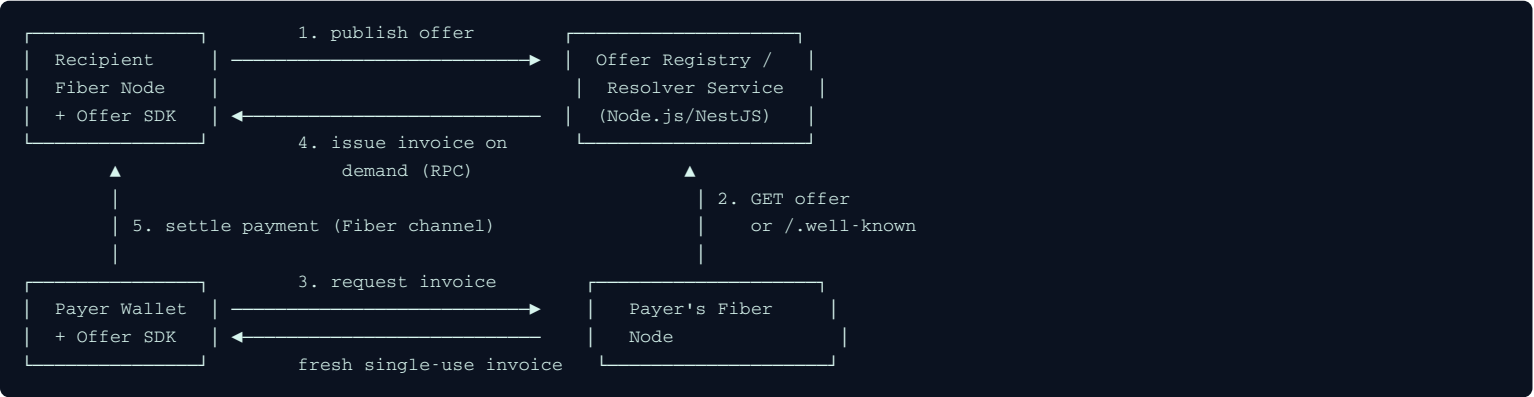
5.1 What Fiber Offers is

Fiber Offers introduces two new primitives on top of existing Fiber invoices:

- 1. **Offer** — a static, signed, reusable payment descriptor. It encodes who to pay, what asset(s) are accepted, optional amount bounds, optional recurrence terms, and how to reach the recipient (directly or via a resolver). It contains no payment hash and is never "consumed."
- 2. **Invoice Request / Resolution** — the step where a payer's wallet takes an Offer and asks for a fresh, single-use Fiber invoice to actually execute the payment. This can happen over HTTP (MVP, LNURL-style) or natively over Fiber's peer messaging (stretch goal, BOLT12-style).

A third, optional layer sits on top: **Fiber Address**, a human-readable identifier (`alice@fiberpay.dev`) that resolves to an Offer via a well-known HTTP path, mirroring how Lightning Address sits on top of LNURL-pay today.

5.2 High-level architecture



6. Core Concepts

6.1 Offer lifecycle

Stage	Description
Create	Recipient's node/app generates an Offer: static metadata + signature over that metadata using the recipient's Fiber node key.
Publish	The Offer is encoded (bech32m-style string + QR) and shared once — on a website, invoice PDF, printed card, or NFC tag.
Resolve	A payer's wallet reads the Offer and requests a fresh invoice: either via the resolver service's HTTP endpoint (MVP) or a direct Fiber peer message (stretch).
Pay	The wallet pays the returned invoice exactly as it would any normal Fiber invoice today — no change to settlement.
Repeat	The same Offer can be resolved and paid again immediately, or on a recurrence schedule, without being regenerated.

6.2 Recurrence model

An Offer may declare a recurrence schedule (e.g. monthly, weekly, custom interval + count). When present, a compatible wallet can present an "approve recurring payment" UI once, then automatically resolve and pay on each due date up to a user-defined spending cap — without further manual approval, unless the cap is exceeded.

6.3 Fiber Address

A Fiber Address such as `leo@loavix.app` resolves via `https://loavix.app/.well-known/fiberoffer/leo` to a canonical Offer, exactly mirroring the LNURL-pay well-known pattern that powers Lightning Addresses today. This is the fastest path to a human-friendly "pay me" identity on Fiber.

7. User Stories & Use Cases

ID	As a...	I want to...	So that...
US-1	Merchant	publish one static QR code at checkout	I don't need a screen generating a new invoice per customer
US-2	Subscriber	approve a recurring Offer once	I'm billed automatically without repeating the approval flow
US-3	Freelancer (Loavix)	send a client a Fiber Address / Offer link	they can pay any invoice I issue without me re-sharing a code
US-4	Wallet developer	drop in an SDK component	my app supports Offers without writing protocol code
US-5	Content creator	publish a static "tip jar" Offer	fans can pay me repeatedly from one link/QR
US-6	Node operator	stay briefly offline	my published Offer can still be resolved via the resolver's cached last-known state, or gracefully queue/retry

8. Feature Set

8.1 MVP (must-ship for hackathon submission)

- **MUST** Offer schema v1 (spec document, versioned, JSON + compact encoded string)
- **MUST** Offer creation library (Node.js/TypeScript, wraps existing Fiber node RPC)
- **MUST** Resolver service (NestJS) exposing `/.well-known/fiberoffer/:id` and a direct `/offers/:id/invoice` endpoint
- **MUST** Offer resolution client (fetch Offer → request invoice → return standard Fiber invoice)
- **MUST** QR + copy-link React/React Native component for displaying an Offer
- **MUST** End-to-end demo: create Offer → scan → pay twice from two different wallet sessions

8.2 Should-have

- **SHOULD** Recurrence fields in schema + a basic auto-resolve scheduler (Redis/BullMQ, reusing patterns already in use for Loavix auth)
- **SHOULD** Fiber Address resolver (`user@domain`) as a thin layer over the Offer resolver
- **SHOULD** Amount-bounded / "any amount" Offers (tip-jar style)

8.3 Could-have / stretch

- **COULD** Native Fiber peer-message transport for resolution (removes HTTP dependency, closer to true BOLT12 parity)
- **COULD** Payer-key commitment for refund-eligibility proof (mirrors BOLT12's refund guarantee)
- **COULD** Multi-asset Offers (accept either CKB-native UDT or RGB++ asset for the same Offer)

8.4 Won't-have (this cycle)

- **WON'T** Full BOLT12 blinded-path privacy parity
- **WON'T** On/off-ramp or fiat settlement
- **WON'T** Mainnet deployment / production security audit

9. User Flows

9.1 Flow: Merchant publishes a static Offer

1. Merchant opens the demo wallet/app and taps "Create Offer."
2. Sets optional amount range, asset, and description ("Coffee Shop Checkout").
3. SDK calls the local Fiber node to fetch node identity, builds and signs the Offer payload, and registers it with the resolver service.
4. App renders a static QR + shareable link. This is displayed once, permanently, at the register.

9.2 Flow: Customer pays the static Offer

1. Customer scans the QR with any Offer-compatible wallet.
2. Wallet resolves the Offer via the resolver's HTTP endpoint, optionally prompting for amount if unspecified.
3. Resolver asks the merchant's node (via RPC) to mint a fresh, single-use Fiber invoice bound to that amount.
4. Wallet pays the invoice through the existing Fiber channel path — no protocol change to settlement.
5. The same QR remains valid for the next customer.

9.3 Flow: Recurring billing (Loavix use case)

1. Loavix issues a client an Offer with a monthly recurrence + fixed stablecoin amount.
2. Client's wallet shows one approval screen: "Approve \$X/month to Loavix, cap \$Y."
3. On each due date, the scheduler (Redis/BullMQ) triggers resolution + payment automatically, up to the cap.
4. Client can revoke the standing approval at any time from their wallet.

10. Success Metrics

Metric	Hackathon target
Working end-to-end demo	Create → publish → resolve → pay, twice, from a single static Offer
Recurrence proof-of-concept	At least one simulated recurring cycle triggers and settles automatically
Reusability	SDK usable by a second, independent demo app with under ~30 minutes of integration
Documentation quality	Spec + README sufficient for another Fiber team to implement a compatible resolver independently
Real-world adoption signal	Loavix integrates Fiber Offers for at least one live billing flow post-hackathon

11. Two-Week Delivery Plan

Days	Focus
1-2	Finalize Offer schema v1, write spec doc, set up devnet + repo scaffolding
3-5	Build offer creation library + resolver service (create, publish, fetch endpoints)
6-8	Build resolution client + invoice-minting RPC integration with Fiber node
9-10	Build QR/link SDK components (React + React Native); wire up first end-to-end demo
11-12	Add recurrence scheduler (Redis/BullMQ) + Fiber Address resolver
13	Integrate with Loavix as the real dogfood consumer; polish demo script
14	Record video, write technical breakdown, finalize submission deliverables

12. Hackathon Deliverables Mapping

Required deliverable	How this project satisfies it
Project summary	Section 1 of this PRD, condensed for submission
Selected category	Wallet and Payment UX Infrastructure
Repository, fully open-sourced	Monorepo: schema/spec, resolver service, SDK, demo app
Demo link / runnable instructions + hosted demo	Resolver deployed to a public devnet-facing host; demo wallet app hosted
Video demonstration	End-to-end flow: create → publish → pay twice → recurring cycle
Technical breakdown	Separate Technical Requirements Document + FRD (this package)
Explanation of infrastructure gap addressed	Section 2, Problem Statement
Future roadmap	Section 8.3/8.4 stretch items + Community Fund DAO grant follow-on potential

13. Risks & Mitigations

Risk	Mitigation
Two weeks is tight for a new protocol + SDK + demo	MVP scoped tightly to HTTP-resolver transport; native peer-message transport deferred to stretch
Fiber node RPC surface may not expose everything needed for on-demand invoice minting	Validate RPC capabilities on Day 1-2 during devnet setup; fall back to a thin sidecar service wrapping the node if needed
Judges may see this as "just LNURL/BOLT12 ported"	Emphasize real dogfooding via Loavix, multi-asset/CKB-specific extensions, and reusability for the wider ecosystem
Recurrence/auto-pay raises security concerns (unattended spending)	Hard spending caps, explicit revocation, and a visible approval screen are non-negotiable parts of the spec, not optional add-ons

Appendix A: Competitive / Comparative Landscape

Capability	Lightning BOLT11	Lightning BOLT12	Fiber (today)	Fiber Offers (this project)
Reusable payment code	No	Yes	No	Yes
Recipient can be briefly offline	No	Partial (blinded paths)	No	Partial (resolver-mediated, MVP)
Native recurring payments	No	Yes	No	Yes
Human-readable identifier	No (LNURL add-on)	Via BIP-353	No	Yes (Fiber Address)
Multi-asset support	No (BTC only)	No (BTC only)	Yes (native)	Yes (inherits Fiber's multi-asset model)

Appendix B: Glossary

Term	Definition
Fiber Network	CKB's payment-channel network for fast, low-cost off-chain payments, supporting multiple assets including RGB++ and CKB-native UDTs.
Invoice	Fiber's existing single-use payment request, analogous to a Lightning BOLT11 invoice.
Offer	This project's new, reusable, static payment descriptor — analogous to a Lightning BOLT12 offer.
Resolver	The HTTP service that turns a published Offer into a fresh Invoice on demand.
Fiber Address	A human-readable identifier (<code>user@domain</code>) that resolves to an Offer, analogous to a Lightning Address.
Recurrence	An optional schedule attached to an Offer enabling automatic repeated payments up to a spending cap.
UDT	User-Defined Token — CKB's native token standard, one of the asset types Fiber and Fiber Offers support.

End of Product Requirements Document. See the companion Technical Requirements Document and Functional Requirements Document for implementation-level detail.